

# Informe Técnico — Red Social sobre Cluster de Base de Datos Distribuido

Sistemas Distribuidos · Proyecto final

Autores: Victor Hugo Pacheco Fonseca (C411) · José Agustín del Toro (C412)

---

## 1 1. Introducción

El proyecto implementa una **red social** (usuarios, posts, follows) cuya capa de datos es un **cluster de base de datos distribuido, jerárquico y tolerante a fallos**. El objetivo es garantizar disponibilidad y no pérdida de datos ante caídas de nodos, manteniendo una arquitectura sencilla y reproducible.

El sistema se compone de:

- **Nodos de estado** (backend/Nodes/): cada uno con su propio Postgres; coordinan liderazgo, replicación y tolerancia a fallos.
  - **API Gateway** (backend/services/): servicio *stateless* que encapsula los endpoints de negocio consumidos por el frontend y los traduce a llamadas al cluster.
  - **Frontend** (Frontend/): aplicación web.
- 

## 2 2.1 Arquitectura y roles

Topología **jerárquica de dos niveles**:

- **follower**: réplica de solo lectura; conoce a su subleader y aplica el WAL que recibe.
- **subleader**: líder de su PC (NODE\_PC\_ID). Punto de contacto del API Gateway: sirve lecturas y reenvía escrituras al leader global.
- **leader (global)**: elegido entre los subleaders; dueño del camino de escritura. Propaga el WAL a sus followers locales y, en multi-PC, a los demás subleaders (Subleader\_manager.relay\_replication).

En un despliegue de **un PC** (caso de la prueba §5) el subleader es inmediatamente el leader global.

El estado del cluster vive en ClusterContext (backend/Nodes/cluster.py): peers, subleader\_id, current\_epoch, last\_applied\_lsn, read\_only. El nodo se modela en node.py (Node, con role, alive, node\_id, node\_pc\_id).

**Despliegue en dos redes Docker** (docker-compose.cluster.yml):

- edge\_net (expuesta): frontend (8080) y API Gateway (7000) — únicos puertos al host.
- cluster\_net (internal: true, **aislada** sin acceso al host ni a internet): nodos + Postgres + gateway (que la usa solo para alcanzar a los nodos). Esto materializa la **frontera de confianza**.

**Volúmenes y persistencia**: cada Postgres tiene su **volumen nombrado** dedicado (dbN\_data:/var/lib/postgresql/data), de modo que el estado de cada nodo es independiente y sobrevive a reinicios del contenedor; make down -v los elimina para volver a un estado limpio reproducible. Los certificados mTLS se montan **solo-lectura** por nodo (./backend/deploy\_certs/node\_N:/app/certs:ro).

---

## 3 2.2 Concurrencia (organización interna multihilo)

Cada nodo es un proceso con **múltiples hilos**:

- El **servidor de control** Flask (`main.py:create_app`) sirve los endpoints sobre HTTPS; atiende peticiones concurrentes.
- **HeartbeatSender** (`HeartbeatSender.py`): hilo *daemon* que emite latidos periódicos (subleader→followers, leader→subleaders) con `threading.Event` para parada limpia.
- **FailureDetector** (`Failure_Detector.py`): hilo *daemon* que vigila el latido del líder y, además, ejecuta **anti-entropía** (un `/sync` incremental por ciclo).
- **Hilo de bootstrap** (`main.py`): ejecuta la reconciliación de liderazgo tras el arranque.

El acceso a Postgres usa **una sesión por operación** (`Database.get_session()`), evitando estado mutable compartido entre hilos a nivel de conexión. Los hilos `start()/stop()` son idempotentes.

---

## 4 2.3 Comunicación

- **Protocolo**: REST sobre **HTTPS** entre todos los componentes.
- **mTLS** (`backend/security/`): una CA raíz (`setup_Security.py`) firma un kit por nodo (`node.crt/.key + ca.crt`). El servidor de cada nodo exige certificado de cliente (`CertManager.get_mtls_context, CERT_REQUIRED`): solo nodos del cluster se comunican entre sí.
- **SNI pinning**: como los nodos se contactan por IP (resuelta vía DNS de Docker) pero el certificado está emitido para el nombre `nodeN.cluster_net`, se usa un `HostHeaderSSLAdapter` (`node_utils.Remote_Comunicate`) que fija el `hostname` esperado del certificado.
- **Timeouts acotados** en todas las llamadas a vecinos (`Remote_Comunicate, General_Endpoint, call_cluster`): una llamada a un nodo caído falla rápido en vez de bloquear el hilo.

El **API Gateway** habla con el cluster también por mTLS (`services/utils.call_cluster`), dirigiéndose siempre al **subleader** vigente.

---

## 5 2.4 Coordinación (elección de líder)

**Algoritmo Bully en dos capas**:

- **Local (por PC)**: ante la caída del subleader, los nodos eligen al de **mayor node\_id** (`cluster.start_election → become_leader`).
- **Global (entre subleaders)**: el subleader, al asumir, descubre otros PCs (`Subleader_manager.discover_global_topology`) y, si no hay leader global, dispara elección entre subleaders (`check_global_leadership`).

**Epoch (term)**: cada vez que un nodo gana el liderazgo global incrementa y "persiste" el epoch (`become_leader`, derivado del WAL durable vía `Database.get_watermark`). El epoch ordena globalmente las escrituras y habilita el *fencing* (§2.6).

**Quórum de liderazgo** (`has_leadership_quorum`): un aspirante a leader global solo asume el rol si alcanza **mayoría de los PCs esperados** (`EXPECTED_PCS`); si no, entra en **modo solo-lectura** (`read_only`). Con `EXPECTED_PCS=1` (un PC) el quórum es trivial.

**Convergencia de arranque** (`ensure_single_leader`): en un arranque simultáneo varios nodos pueden auto-promoverse a subleader a la vez. Tras el bootstrap, cada nodo sondea por `/info` a los nodos vivos de su PC y aplica Bully de forma determinista: el de mayor `node_id` permanece como subleader único y el resto cede a follower (`step_down_to_`

follower) y re-sincroniza. Esto colapsa cualquier multi-líder transitorio a **exactamente un líder**.

---

## 6 2.5 Nombrado

- **Identidad de nodo:** `NODE_ID` (id lógico, criterio de Bully) y `NODE_PC_ID` (PC al que pertenece), por variables de entorno (`Node.from_env`).
  - **Identidad de entradas WAL:** `wal_id = "{node_id}:{epoch}:{lsn}"` (`models/wal.py`), con orden global por la tupla (`epoch, lsn`).
  - **Descubrimiento por DNS:** los nodos comparten el alias `Docker cluster_net_serv`; un nodo enumera a sus pares con `utils.get_ips_for_alias` y obtiene la vista del cluster con `/info`. El gateway resuelve a los nodos por el mismo alias; el subleader vigente, además, **se anuncia** al gateway (`notify_gateway → POST /cluster/subleader`).
- 

## 7 2.6 Consistencia y replicación (sección crítica)

### 7.1 Modelo

**Consistencia eventual con single-writer leader y prevención de split-brain por epoch + quórum.**

- Las escrituras solo las acepta el **leader** (`/db/posts|users|follows` comprueban `is_leader()`; si no, reenvían al leader). El leader asigna `lsn` monótono (`next_lsn`) y registra un WAL con (`epoch, lsn`) (`Save_Wallog`).
- La **replicación** es por WAL lógico: el leader confirma localmente y propaga *best-effort* a los followers (`replicate_to_followers`) y a otros subleaders (`relay_replication`); el subleader receptor re-replica a sus followers (`/replicate`). **Lectura propia garantizada en el leader**; el resto converge de forma eventual.

### 7.2 Orden global y idempotencia

Toda entrada se ordena por la tupla (**epoch, lsn**). Una única comparación lexicográfica (`is_newer`) resuelve a la vez **idempotencia** (ignorar lo ya aplicado) y **fencing** (ignorar a un leader viejo de epoch inferior). El watermark (`current_epoch, last_applied_lsn`) se reconstruye del WAL durable al arrancar (`Database.get_watermark`), de modo que el orden sobrevive a reinicios.

### 7.3 Partición y split-brain — prevención (no solo documentación)

1. **Fencing por epoch:** los heartbeats y `/replicate` llevan epoch. Un nodo **rechaza** mensajes con `epoch < current_epoch` (`/heartbeat, /heartbeat_leader` devuelven 409; `/replicate` ignora vía `is_newer`). Un leader viejo que reaparece tras una partición queda **fenced** automáticamente: sus mensajes son descartados y su `adopt_higher_epoch` lo hace ceder a follower.
2. **Quórum de liderazgo:** una partición minoritaria no alcanza mayoría de subleaders y entra en **solo-lectura** (`read_only`), evitando un segundo leader que acepte escrituras.
3. **Ventana de arranque:** el breve multi-líder posible durante el bootstrap se resuelve de forma determinista por `ensure_single_leader` (§2.4); el perdedor re-sincroniza el estado del ganador.

## 7.4 Reconciliación post-partición / reingreso

Al reintegrarse, un nodo compara su watermark (epoch, lsn) y hace **replay incremental** del WAL faltante vía /sync (sync\_from\_leader; el servidor devuelve todo lo posterior a (from\_epoch, from\_lsn) ordenado por (epoch, lsn)). Las entradas de un epoch perdedor quedan por debajo de las del ganador y son sobrescritas/ignoradas por el orden global. Tras una promoción se reajustan las **secuencias de id** (users/posts/follows) a MAX(id) para no colisionar con filas ya replicadas.

## 7.5 Fuera de alcance (decisión consciente)

No se implementan **ack de quórum en la escritura**, **commit distribuido** ni **replicated state machine**. Consecuencia: una escritura confirmada por el leader pero aún no propagada podría perderse si el leader cae en esa ventana. Se acepta por el modelo de consistencia eventual declarado; la rúbrica §5 (no pérdida de los datos *replicados*) se satisface porque la verificación se hace tras la propagación.

---

## 8 2.7 Tolerancia a fallos

**Nivel de tolerancia objetivo:** fallos *fail-stop* (caída de proceso/nodo), **no bizantinos**. Con un PC de N nodos el sistema tolera la pérdida de N-1 nodos sin perder los datos ya replicados: mientras sobreviva un nodo con el WAL al día, hay disponibilidad de lectura y, tras la reelección, de escritura. La prueba §5 ejerce el caso extremo (de 3 nodos quedan 0 originales) reincorporando un nodo nuevo antes de la última caída. No se persigue tolerancia bizantina ni alta disponibilidad multi-PC con *failover* automático entre PCs (queda como extensión).

- **Detección:** latidos periódicos + FailureDetector con timeout; al expirar, marca al líder caído (mark\_peer\_down: alive=False, role→follower) y dispara reelección.
- **Reelección:** Bully en la capa correspondiente; el nuevo leader sube el epoch (fencing del anterior).
- **Estado alive consistente:** mark\_peer\_down es el punto único de baja; Craft\_Node preserva el alive real, de modo que un **nodo nuevo hereda** qué pares están caídos y no malgasta llamadas. Todas las llamadas a vecinos están protegidas (try/except + timeout): un follower caído nunca aborta una escritura ya confirmada.
- **Join protocol / reingreso:** el nodo nuevo descubre la topología, se registra (/newNode), sincroniza el WAL (/sync) y queda como follower; al caer el resto, es elegido leader.
- **Anti-entropía:** cada follower ejecuta /sync periódico, recuperando WAL perdido (idem-potente por (epoch, lsn)).

**Evidencia (prueba §5, make test):** escenario 3↑ → 2↓ (incluido el leader) → 1 nuevo ↑ → 1↓. Resultado **PRUEBA §5 SUPERADA (PASS=13, FAIL=0)**: checksums idénticos en los 3 nodos iniciales, dos reelecciones con incremento de epoch (1→2→3), nodo nuevo sincronizado al estado completo y **sin pérdida de datos** tras los dos failovers. Logs y checksums en pruebas/evidence/.

---

## 9 2.8 Seguridad

- **CA + mTLS** (setup\_Security.py, CertManager): identidad por certificado para todo nodo; el canal entre nodos exige certificado de cliente (CERT\_REQUIRED) → solo el cluster habla con el cluster.

- **JWT** (api\_gateway): autenticación de usuarios de la red social; rutas /auth y /cluster públicas, el resto exige token válido.
- **bcrypt**: las contraseñas se almacenan hasheadas (el gateway hashea antes de enviarlas al cluster).
- **Mínimo privilegio / aislamiento**: cluster\_net es internal (sin host ni internet); solo el frontend y el gateway exponen puertos (edge\_net). El canal interno de registro del subleader (/cluster/subleader) vive en la red aislada.
- **Gestión y rotación de credenciales**: los certificados mTLS los emite la CA del proyecto (setup\_Security.py) y se montan por nodo en **solo-lectura**; la **rotación** se realiza regenerando el kit (nueva CA/certs) y reconstruyendo (make certs tras borrar deploy\_certs/, luego make up). El **secreto JWT** (JWT\_SECRET\_KEY) es una variable de entorno del gateway, rotatable por despliegue; las contraseñas nunca se guardan en claro (solo su hash bcrypt). *Fuera de alcance*: rotación automática/sin reinicio y un gestor de secretos externo (Vault, etc.).

## 10 Ejecución del Proyecto

**Prerrequisitos:** Docker + Docker Compose v2 y make.

```
git clone <repo> && cd SocialNetwork
make up # certs (auto) + build + 3 nodos + gateway + frontend (UN comando)
make health # verificar salud; converge a 1 leader + 2 followers (~10 s)
# Frontend: http://localhost:8080 · API: https://localhost:7000
make test # prueba obligatoria de tolerancia $5 (genera pruebas/evidence/)
make down # estado limpio
```

Comandos de demo de fallos: make add-node (nodo nuevo), make kill-node N=, make start-node N=, make logs. Video de demostración: ver [video\\_link.txt](#).

### 10.1 Mapa de código (referencia rápida)

| Tema                                             | Dónde                                         |
|--------------------------------------------------|-----------------------------------------------|
| Estado, elección, replicación, fencing           | backend/Nodes/cluster.py                      |
| Subleader / leader global / relay cross-PC       | backend/Nodes/Subleader_manager.py            |
| Heartbeats / detección de fallos / anti-entropía | HeartbeatSender.py, Failure_Detector.py       |
| Endpoints de control y datos                     | backend/Nodes/routes/                         |
| WAL y persistencia                               | models/wal.py, DataBase.py                    |
| API Gateway (push subleader, JWT)                | backend/services/                             |
| Seguridad (CA, mTLS)                             | backend/security/                             |
| Despliegue (2 redes, 1 comando)                  | docker-compose.cluster.yml, Makefile          |
| Prueba \$5 + evidencia                           | pruebas/test_tolerancia.sh, pruebas/evidence/ |